

NVDLA on PetaLinux 2024.1

Correctness, latency, throughput, and monitored energy at a glance

100 / 100

LeNet stability

One boot; no module reload or PL reset

246 / 246

ResNet-50 hardware layers

Exact match to source-built nv_small VP golden

1.34x

Four-core CPU INT8 advantage

378.7 ms CPU INT8 vs 507.7 ms NVDLA on ResNet-50

1.92x

NVDLA cold-deployment advantage

1.336 s NVDLA vs 2.566 s CPU INT8 on ResNet-50

-62.3%

NVDLA incremental energy

0.134 J NVDLA vs 0.356 J CPU INT8 on ResNet-50

20 x 2

Input-sensitivity control

Balanced image sets; three fresh boots per model

Comparison boundary: nv_small NVDLA INT8 is compared with FP32 and independently quantized QDQ INT8 ONNX Runtime using one, two and four Cortex-A53 cores. Equal nominal precision does not imply identical quantization or graph transformation. The performance dataset contains 140 fresh-boot sessions; six supplementary sessions test 20 images per model.

Correctness before performance

Layered acceptance

OK

Pinned inputs

OK

ABI and build

OK

Verified nv_small
VP

OK

Driver probe

OK

GEM mapping

OK

IRQ + engine
completion

OK

Exact tensor +
repeat

LeNet / MNIST

Fast repeat oracle

INPUT
1 x 1 x 28 x 28LOADABLE
0.446 MBHARDWARE LAYERS
10ENGINE MIX
4 Conv / 4 SDP / 2 PDPVP
Exact outputBOARD
Exact; 100/100 repeats

Output: 0 2 0 0 0 0 0 124 0 0

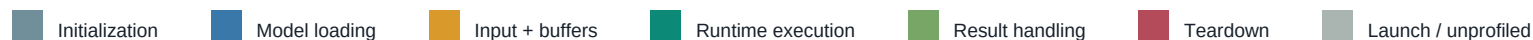
ResNet-50

Large multi-engine graph

INPUT
1 x 3 x 224 x 224LOADABLE
25.77 MBHARDWARE LAYERS
246ENGINE MIX
114 Conv / 130 SDP / 2 PDPVP
246/246; golden establishedBOARD
246/246; exact hash

Output: 1,000 signed values; SHA-256 842d34f...

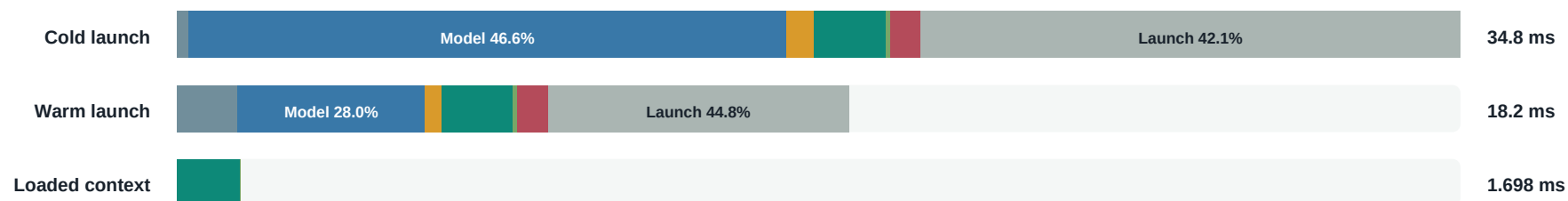
Where does NVDLA deployment time go?



LeNet

bar length is relative to this model's cold launch-to-exit time

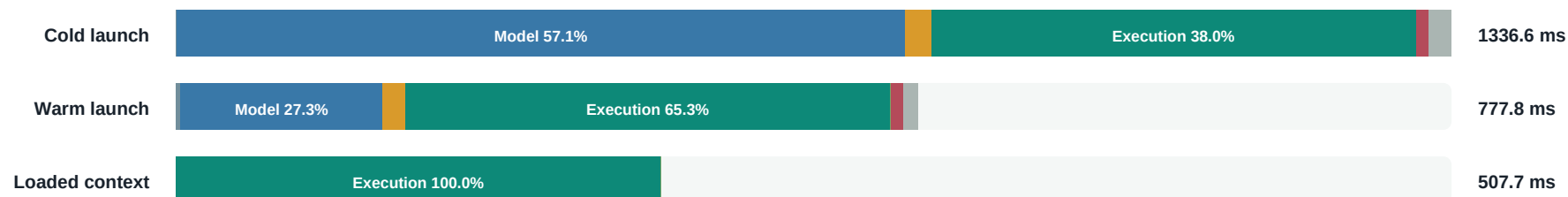
Warm runtime execution share 10.7% | loaded context 1.698 ms



ResNet-50

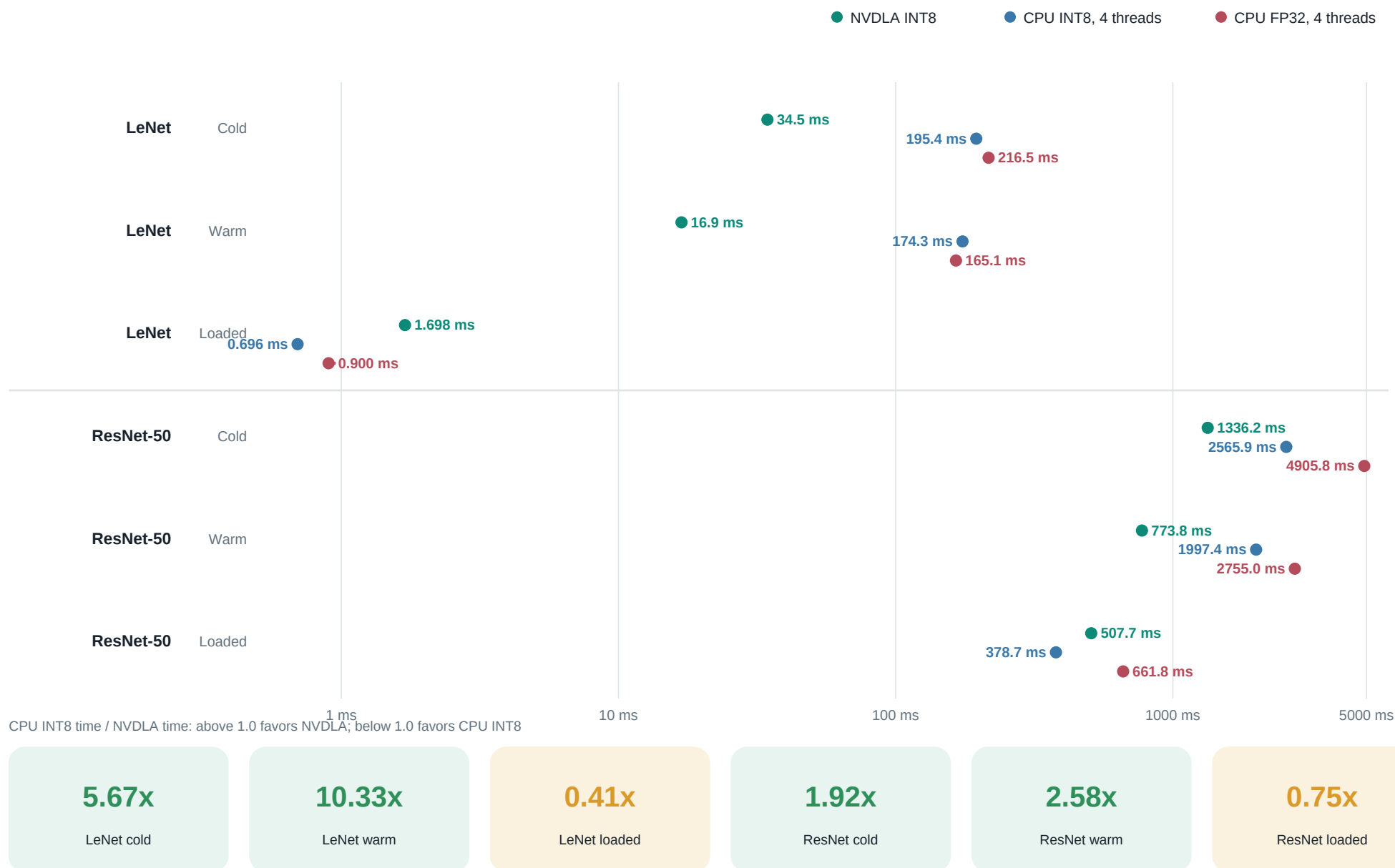
bar length is relative to this model's cold launch-to-exit time

Warm runtime execution share 65.3% | loaded context 507.7 ms



Timing boundary: cold and warm bars span process launch to exit. Loaded context reuses the model and buffers; its blocking runtime execution includes userspace dispatch, ioctl/KMD scheduling, accelerator work, IRQ completion, and emulator tasks.

Three deployed stacks across each timing boundary



Does execution time depend on the image?

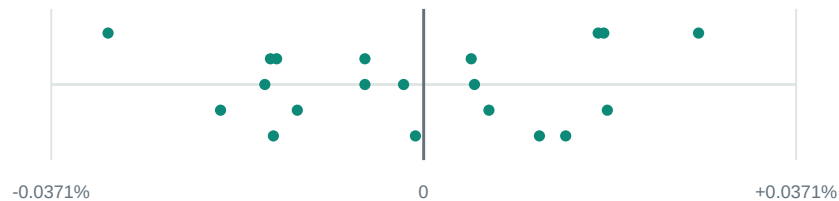
Execution-validity result: every input produced a repeat-stable output, increased IRQ activity, and completed without a bad kernel pattern.

LeNet / MNIST

20 inputs x 30 measured executions; loaded model and buffers reused

Per-input median execution deviation

model-specific horizontal scale



Observed range across the 20 medians: 0.0589%

Input decode and file I/O are outside this execution interval.

1.699 ms

Median execution

0.002 ms

Prepared input copy

20 / 20

Stable outputs

16 / 20

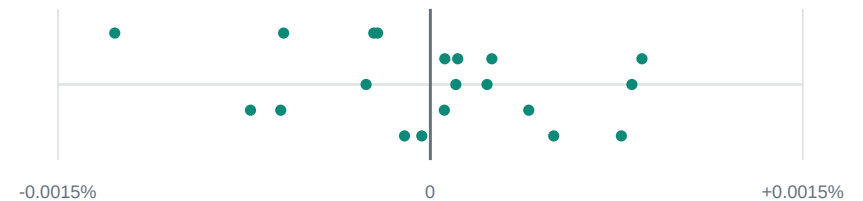
Recorded top-1

ResNet-50 / Imagenette

20 inputs x 15 measured executions; loaded model and buffers reused

Per-input median execution deviation

model-specific horizontal scale



Observed range across the 20 medians: 0.0022%

Input decode and file I/O are outside this execution interval.

507.8 ms

Median execution

0.171 ms

Prepared input copy

20 / 20

Stable outputs

17 / 20

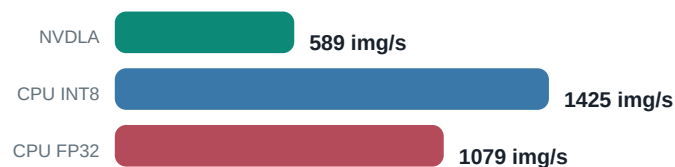
Recorded top-5

Interpretation: class accuracy is descriptive model-quality context from a small curated set. Hardware acceptance is based on repeat-stable output, IRQ progress, and clean kernel logs.

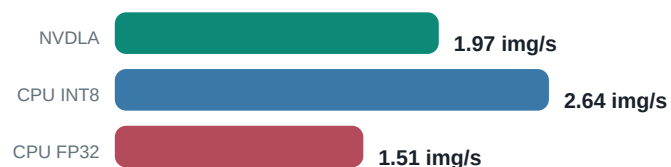
What changes between LeNet and ResNet-50?

Loaded-context throughput

LeNet



ResNet-50



Workload scale and operation mix

LeNet

NVDLA: 0.446 MB loadable | 10 hardware layers

CPU ONNX: 1.73 MB FP32 / 0.45 MB INT8



ResNet-50

NVDLA: 25.77 MB loadable | 246 hardware layers

CPU ONNX: 102.48 MB FP32 / 26.09 MB INT8



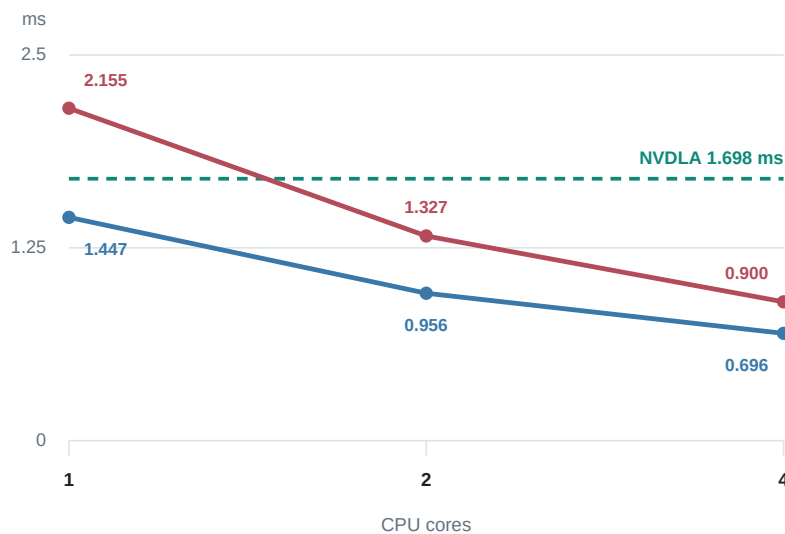
ResNet-50 has **24.6x** more hardware layers and a **57.8x** larger loadable. Its larger execution graph amortizes fixed submission and interrupt costs.

How does execution latency scale with CPU cores?

— CPU INT8 — CPU FP32 - - - NVDLA INT8 reference

LeNet

session-median loaded execution; lower is better

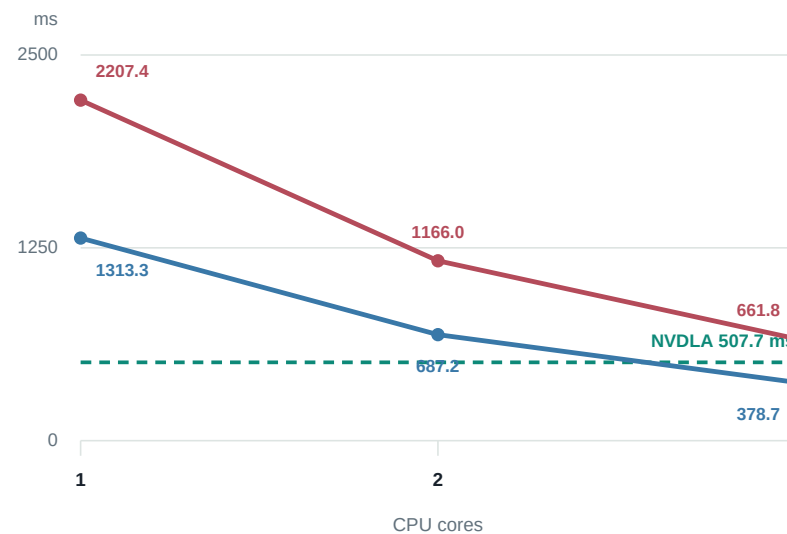


1 to 4 cores: 2.08x INT8 | 2.39x FP32

NVDLA beats one-thread FP32; CPU INT8 remains faster.

ResNet-50

session-median loaded execution; lower is better



1 to 4 cores: 3.47x INT8 | 3.34x FP32

NVDLA beats one- and two-core CPU INT8; four cores are faster.

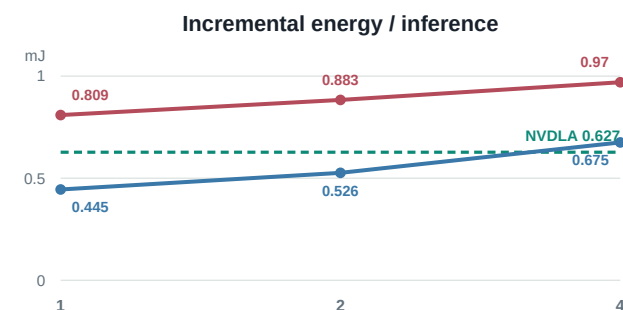
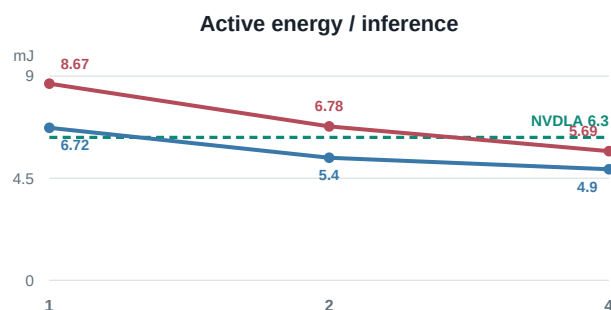
Interpretation: ResNet-50 exposes useful CPU parallelism and places NVDLA between two and four Cortex-A53 cores. LeNet is dominated by fixed costs, so adding cores gives weaker scaling and does not create an accelerator advantage over CPU INT8.

What happens to energy as CPU cores are added?

— CPU INT8 — CPU FP32 - - - NVDLA INT8 reference

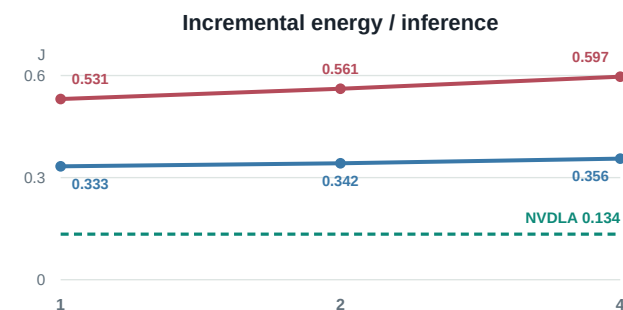
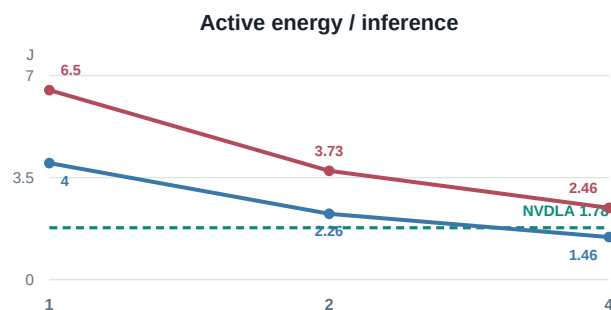
LeNet

x-axis: 1, 2 and 4 CPU cores



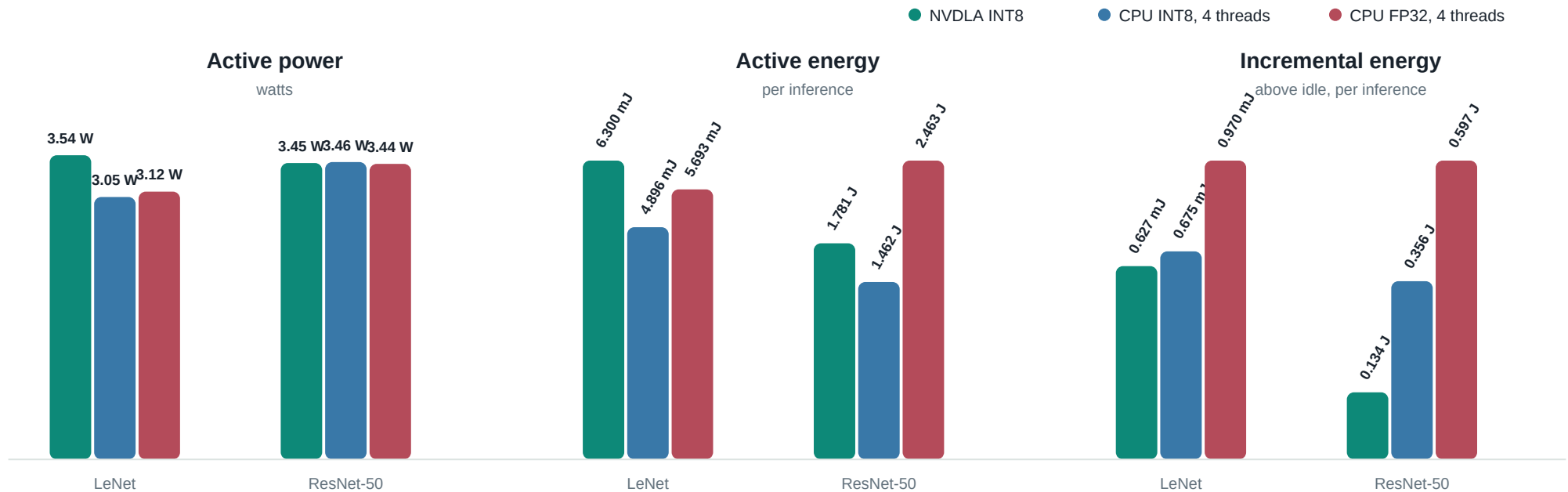
ResNet-50

x-axis: 1, 2 and 4 CPU cores



Result: more CPU cores reduce active energy because the shorter execution outweighs higher power. Incremental CPU energy rises slightly. On ResNet-50, NVDLA uses less active energy than one- or two-core CPU INT8 and substantially less incremental energy than every CPU configuration.

Monitored PS + PL rails during inference



LeNet

+28.7% NVDLA active energy
-7.1% incremental

Change is NVDLA relative to CPU INT8. Active includes the monitored platform; incremental subtracts driver-loaded idle.

ResNet-50

+21.8% NVDLA active energy
-62.3% incremental

Change is NVDLA relative to CPU INT8. Active includes the monitored platform; incremental subtracts driver-loaded idle.

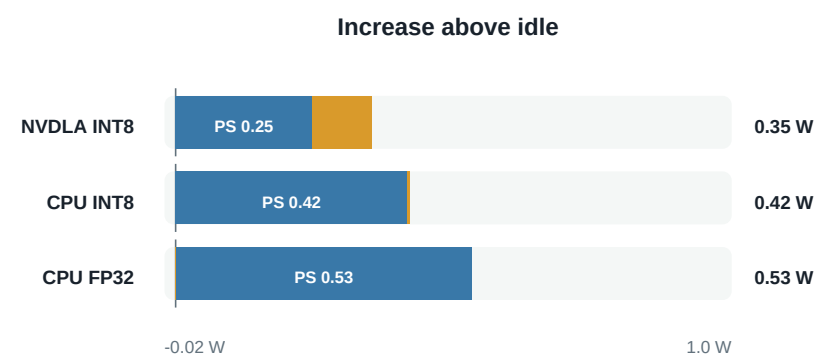
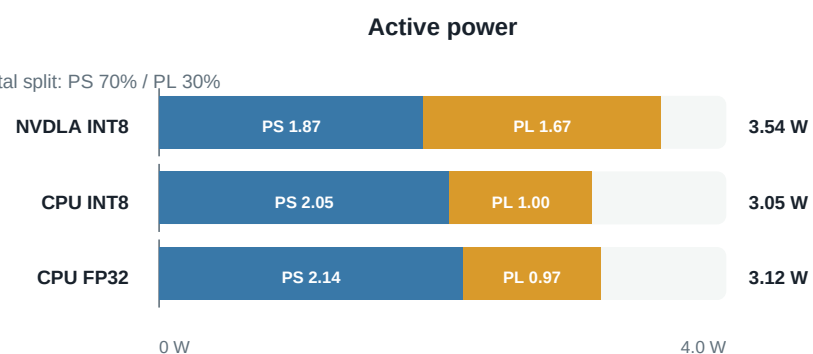
How do PS and PL contribute?

■ PS rails ■ PL rails

Active = complete monitored board state | Incremental = active minus stack-specific idle

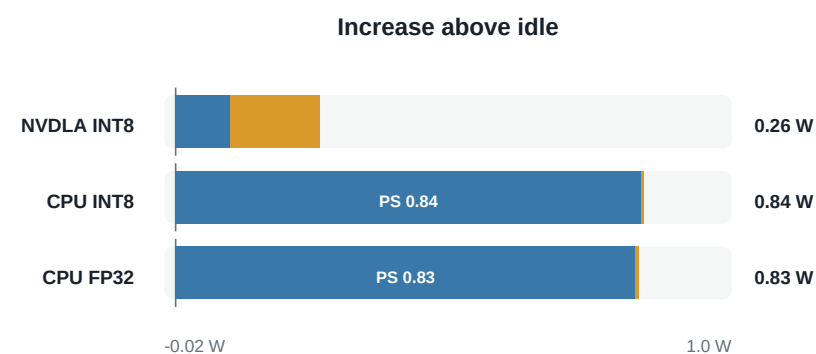
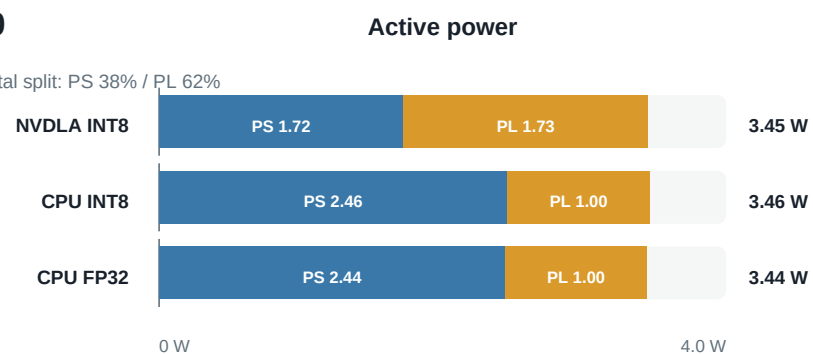
LeNet

NVDLA incremental split: PS 70% / PL 30%



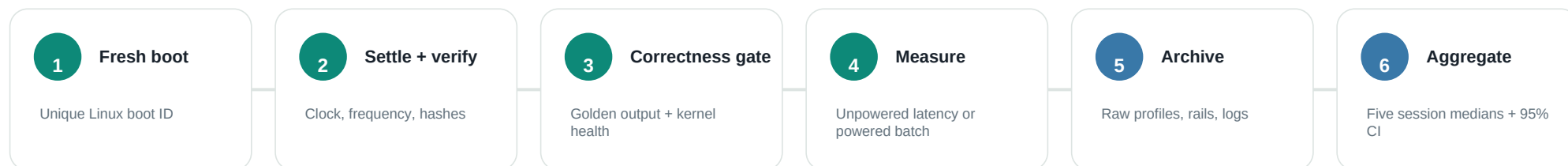
ResNet-50

NVDLA incremental split: PS 38% / PL 62%



Reading the split: CPU inference activity is overwhelmingly on PS rails. NVDLA distributes activity across PS and PL; for ResNet-50, PL contributes about 62% of its incremental power. Tiny negative PL deltas are retained and indicate the sensor/idle-subtraction noise floor.

What was collected, and how to read it



Metric inventory

Correctness	Output hashes, exact tensors, engine sequence, IRQ deltas, repeat failures	Workload	Input shape, loadable/model size, HWL count, per-engine operation count
Latency	Cold deployment, warm deployment, loaded-context execution, runtime phases	Throughput	Images/s for every timing boundary; CPU/NVDLA latency ratios
Power	18 PS + PL rails, idle/active/incremental watts, integrated energy/inference	Environment	Kernel, binary hashes, clock, CPU affinity/frequency/governor, boot ID
Statistics	Raw samples, mean/median, spread, percentiles, retained outliers, bootstrap CI	Evidence	Serial, dmesg, runtime profiles, sensor samples, manifests, selection hashes

Interpret with: one ZCU102 and nv_small implementation; NVDLA INT8 versus independently quantized CPU INT8 plus FP32; monitored rails rather than wall input; five boots per performance/scaling cohort and three per input-sensitivity model.